

Machine Learning:

Classifier Evaluation

Xiuxia Du, Ph.D.

Department of Bioinformatics and Genomics
University of North Carolina at Charlotte

Outline

- Why do we need to evaluate classifiers?
- Contingency table and related core metrics
- ROC and AUC
- Precision-Recall curve
- Overfitting
- Regularization
- Estimation of generalization performance

Why Do We Need to Evaluate Classifiers?

- **Training performance is misleading:** A model can memorize training data (overfitting) and fail on new data.
- **Different mistakes have different consequences:** False negatives (missed positives) vs false positives (false alarms) matter differently by application.
- **Accuracy alone can be dangerously wrong:** On imbalanced data, a useless model can still achieve very high accuracy.

Why Do We Need to Evaluate Classifiers?

- **Classifiers require a decision threshold:** Probabilities must be converted into labels; threshold choice controls the precision–recall tradeoff.
- **We must compare models fairly:** Evaluation provides a consistent basis to choose the best model for the task.
- **Scientific and real-world accountability:** Metrics, confusion matrices, ROC/PR curves, and cross-validation justify model reliability.

Bottom line: Evaluation tells us how a classifier will behave on unseen data and whether its errors match the real-world goal.

Contingency Table and Related Metrics

From class probability to class label

- Most classifiers output a score or probability:

$$\hat{p}(y = 1|x)$$

- To turn that into a class label, we choose a threshold t :

$$y = \begin{cases} 1 & \text{if } \hat{p} \geq t \\ 0 & \text{otherwise} \end{cases}$$

- Once we choose the threshold t , we can construct a 2×2 **contingency table**.

Contingency table

- Also known as a *confusion matrix* or an *error matrix*.
- Binary classification

	Predicted Negative (0)	Predicted Positive (1)
Actual Negative (0)	TN	FP
Actual Positive (1)	FN	TP

- Key Idea:
 - TN = correctly predicted negatives
 - FP = false alarms
 - FN = missed positives
 - TP = correctly predicted positives

Contingency table

- From these four numbers come a number of metrics:
 - sensitivity, recall
 - specificity
 - precision
 - accuracy
 - F_1 score
 - and more
- **A confusion matrix describes classifier behavior at one particular decision threshold.**
- As we change the probability threshold t , the contingency table changes and these metrics change too.

Core Metrics

	Pred. 0	Pred. 1	Total	Metric	Metric
Act. 0	TN	FP	TN+FP	$Sp = \frac{TN}{TN + FP}$	$FPR = \frac{FP}{FP + TN}$
Act. 1	FN	TP	FN+TP	$R = Se = TPR = \frac{TP}{TP + FN}$	
Total	TN+FN	FP+TP	TP+TN+FP+FN	$A = \frac{TP + TN}{TP + TN + FP + FN}$	
Metric		$P = \frac{TP}{TP + FP}$		$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	
Metric		$FDR = \frac{FP}{FP + TP}$			

Sp = Specificity; Se = Sensitivity; R = Recall; P = Precision; A = Accuracy

Core Metrics

- **Accuracy**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

[V] Good when classes are balanced.

[X] Bad when classes are imbalanced. A 98%-negative dataset allows 98% accuracy by predicting everyone negative.

- **Precision (Positive Predictive Value)**

$$\text{Precision} = \frac{TP}{TP + FP}$$

Interpretation: When the model predicts positive, how often is it right?

Core Metrics

- **Recall (Sensitivity or True Positive Rate)**

$$\text{Recall} = \frac{TP}{TP + FN}$$

Interpretation: Out of all true positives, what fraction did we detect? High sensitivity means few false negatives. Important in screening and rare-event detection.

- **Specificity (True Negative Rate)**

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Interpretation: Out of all true negative, what fraction did we correctly reject? High specificity means few false positives.

Core Metrics

- **F1 Score**

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- To get a good F1 score, both the precision and recall need to be high.
- F1 Score is especially useful in imbalanced datasets where a low value in either precision or recall can significantly lower the F1 score.

Core Metrics

- Specificity = True Negative Rate (TNR)
- False Positive Rate (FPR)

$$FPR = \frac{FP}{FP + TN} = 1 - \frac{TN}{FP + TN} = 1 - \text{Specificity}$$

- True Positive Rate (TPR) = Recall = Sensitivity
- False Discovery Rate (FDR)

$$FDR = \frac{FP}{FP + TP} = 1 - \text{Precision}$$

ROC and AUC

ROC

- As the threshold t is changed:

$$1.0 \rightarrow 0.9 \rightarrow 0.8 \rightarrow \dots \rightarrow 0.2 \rightarrow 0.1 \rightarrow 0.0$$

we calculate the following at each t :

- $\text{TPR} = \text{Sensitivity} = \text{Recall}$
- $\text{FPR} = 1 - \text{Specificity}$

- Then plot: TPR vs FPR
- This produces the **Receiver Operating Characteristics (ROC)** curve. Each point on the ROC curve corresponds to one threshold t

ROC

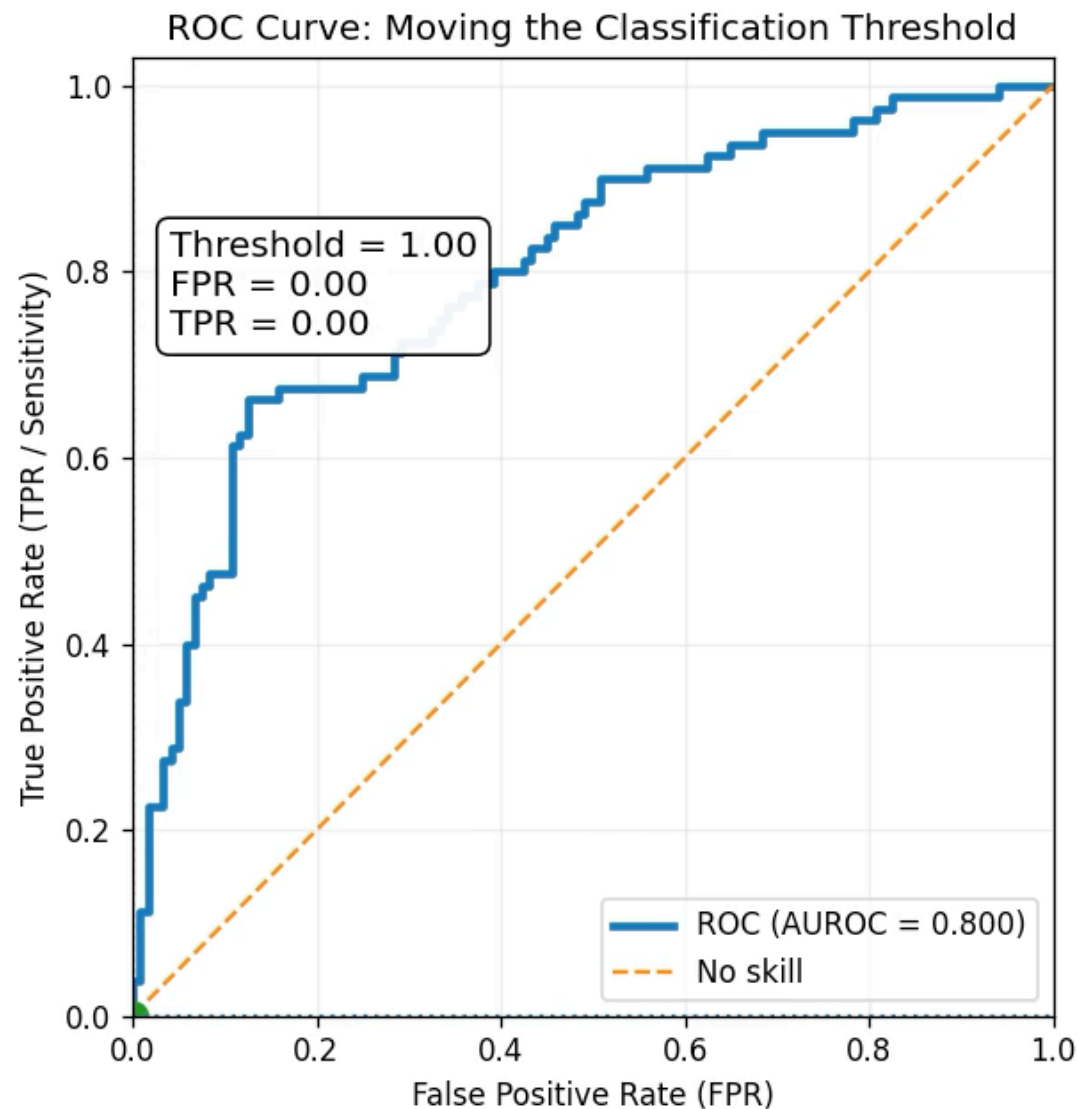
	Pred. 0	Pred. 1	Total		Metric	Metric
Act. 0	TN	FP	TN+FP		$Sp = \frac{TN}{TN + FP}$	$FPR = \frac{FP}{FP + TN}$
Act. 1	FN	TP	FN+TP		$R = Se = TPR = \frac{TP}{TP + FN}$	
Total	TN+FN	FP+TP	TP+TN+FP+FN		$A = \frac{TP + TN}{TP + TN + FP + FN}$	
Metric		$P = \frac{TP}{TP + FP}$			$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	
Metric		$FDR = \frac{FP}{FP + TP}$				

Y-axis

X-axis

Sp = Specificity; Se = Sensitivity; R = Recall; P = Precision; A = Accuracy

ROC (animation)



Confusion Matrix

	Predicted -	Predicted +
Actual -	TN 120	FP 0
Actual +	FN 80	TP 0

Threshold 1.00 | Sensitivity 0.000 | Specificity 1.000 | Precision 0.000 | AUROC 0.800

ROC curve and AUROC stay fixed; moving the threshold changes the operating point and threshold-dependent metrics.

Contingency Table and ROC

ROC summarizes the behavior of many threshold-specific confusion matrices.

How to Interpret the ROC Curve?

- If the ROC curve is a diagonal line, i.e., $TPR = FPR$, the classifier is completely random.
- Consider a point on the diagonal line such as $FPR = 0.40$ and $TPR = 0.40$. The classifier is classifying 40% of actual positives as positives, but it is also classifying 40% of actual negatives as positives.

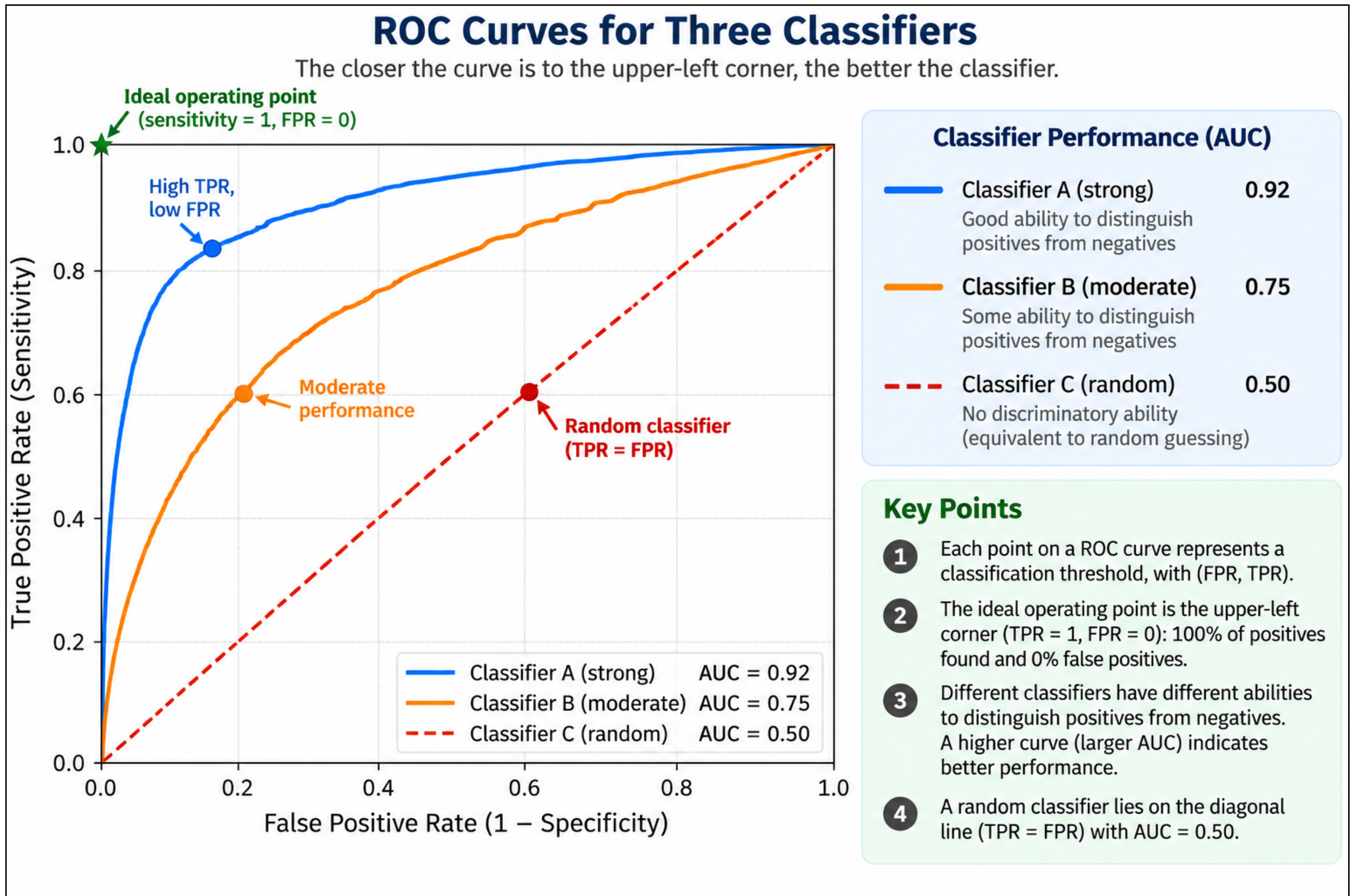
When TPR is increased to 0.50, FPR is also increased to 0.50.

- The classifier treats the two groups identically. Knowing the classifier's score provides no useful information for separating positive from negatives.
- The classifier has no ability whatsoever to distinguish positives from negatives. Its scores are unrelated to the true class.

What is AUC?

- AUC = Area Under the ROC Curve
- It is a single number summary of the ROC curve.
- Interpretation
 - AUC = 0.5: random guessing
 - AUC = 1.0: perfect prediction
 - AUC < 0.5: model is backwards.
- **AUROC is the probability that a randomly selected positive sample receives a higher model score than a randomly selected negative sample.**
- In logistic regression, model score is $p(Y = 1|X)$.

ROC and AUC



Precision-Recall

Imbalanced Dataset

Example: A dataset has 100 total subjects, among which 90 are positive and 10 are negative.

	Predicted Neg. (0)	Predicted Pos. (1)	Total	Metric
Actual Neg. (0)	5	5	10	$S = \frac{5}{10} = 0.5$
Actual Pos. (1)	10	80	90	$R = \frac{80}{90} = 0.88$
Total	15	85	100	$A = \frac{80 + 5}{100} = 0.85$
		$P = \frac{80}{85} = 0.94$		$F1 = 0.91$

Imbalanced Dataset

Example: A dataset has 100 total subjects, among which 9 are positive and 91 are negative.

	Pred. 0	Pred. 1	Total	Metric
Actual 0	90	1	91	$Sp = \frac{90}{90 + 1} = 0.99$
Actual 1	8	1	9	$R = Se = TPR = \frac{1}{9} = 0.11$
Total	98	2	100	$A = \frac{90 + 1}{100} = 0.91$
		$P = \frac{1}{2} = 0.5$		$F1 = 0.18$

misleading high accuracy

Imbalanced Dataset

Example: Suppose we have 1,000 patients, among which 990 do NOT have a disease (negative = 0) and 10 DO have the disease (positive = 1).

- 990 do NOT have the disease (negative = 0) and 10 DO have the disease (positive = 1).
- A dumb classifier can predict “No disease” for everyone. so:

$$TP = 0, TN = 990, FN = 10, FP = 0$$

- Accuracy will be:

$$Accuracy = \frac{TP + TN}{1000} = \frac{0 + 990}{1000} = 0.99$$

Accuracy is meaningless here, because the model is just exploiting the majority class.

- Recall will be:

$$Recall = \frac{TP}{TP + FN} = \frac{0}{0 + 10} = 0$$

It detects 0% of true positives.

Imbalanced Dataset

- One class is much rarer than the other.
- Metrics like accuracy can become misleading when one class dominates.
- Use the following instead:
 - Precision and recall
 - F1 score
 - Precision-Recall curve AUC

Precision-Recall Curve

- Precision-Recall Curve
 - x-axis: Recall
 - “Of all real positives, how many did we catch?”
 - y-axis: Precision
 - “Of all predicted positives, how many were correct?”

Precision-Recall Curve

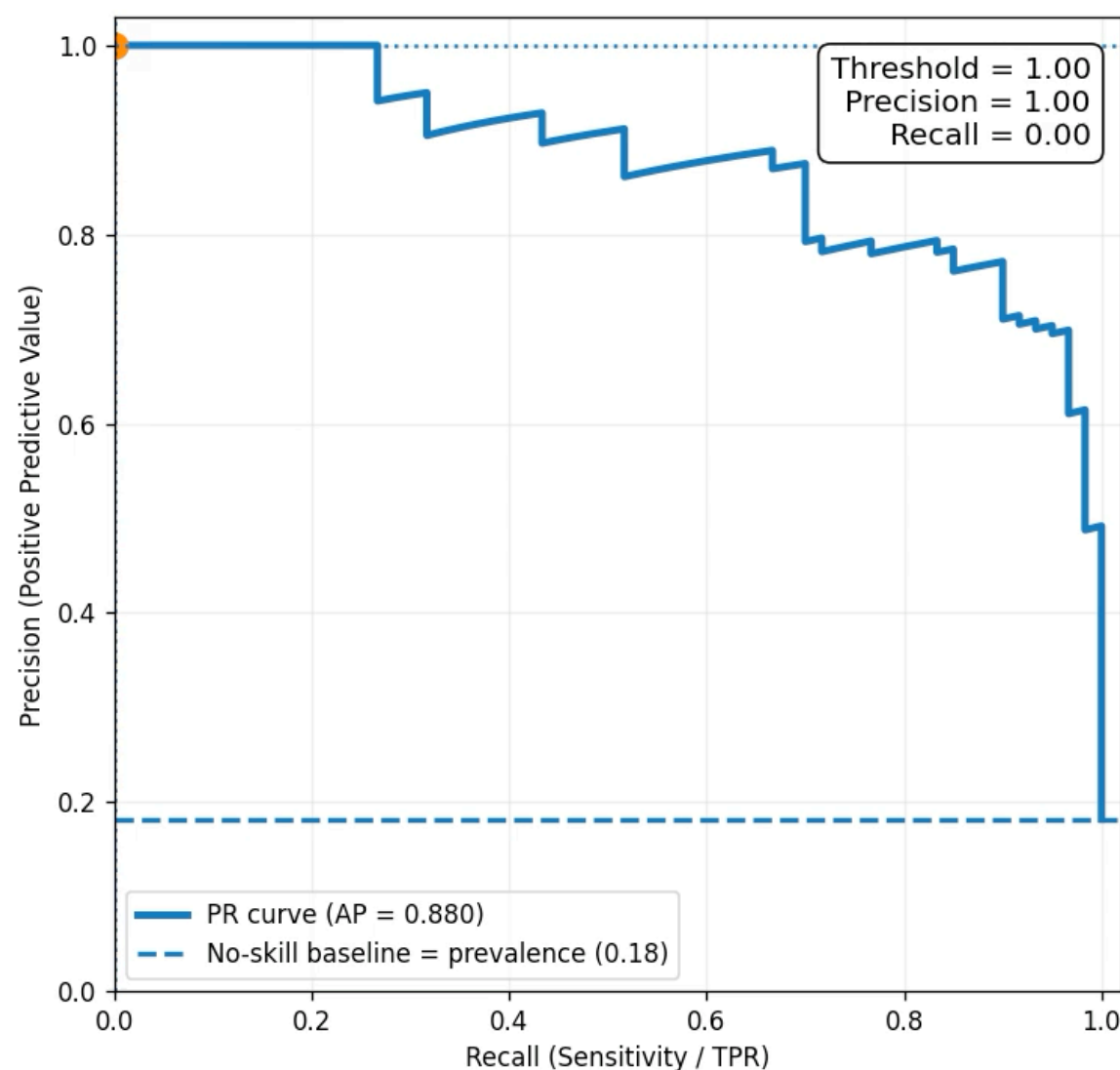
	Pred. 0	Pred. 1	Total	Metric	Metric
Act. 0	TN	FP	TN+FP	$Sp = \frac{TN}{TN + FP}$	$FPR = \frac{FP}{FP + TN}$
Act. 1	FN	TP	FN+TP	$R = Se = TPR = \frac{TP}{TP + FN}$	
Total	TN+FN	FP+TP	TP+TN+FP+FN	$A = \frac{TP + TN}{TP + TN + FP + FN}$	
Metric		$P = \frac{TP}{TP + FP}$		$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	
Metric		$FDR = \frac{FP}{FP + TP}$			
Sp = Specificity; Se = Sensitivity; R = Recall; P = Precision; A = Accuracy					

X-axis

Y-axis

Precision-Recall Curve (animation)

Precision-Recall Curve: Moving the Classification Threshold



Classification Threshold



Threshold = 1.00

Confusion Matrix

	Pred -	Pred +
Actual -	270	0
Actual +	60	0

Performance Metrics

Precision (PPV)	1.000
Recall (Sensitivity)	0.000
Specificity (TNR)	1.000
False Positive Rate	0.000
F1 score	0.000
Accuracy	0.818

Average Precision 0.880

Lower threshold → generally higher recall, but precision may decrease. The PR curve and Average Precision remain fixed.

PR Curve: The Key Tradeoff

Suppose a model outputs a probability such as $P(Y = 1 \mid X)$, and initially we classify a sample as positive when

$$P(Y = 1 \mid X) \geq 0.5$$

- To increase recall, lower the classification threshold. Suppose we change the threshold from 0.5 to 0.3.

Now more samples are classified as positive. This has two important consequences:

$$\text{FN} \rightarrow \text{TP}$$

for some actual positive samples, which increases recall.

But at the same time, some actual negative samples may change from

$$\text{TN} \rightarrow \text{FP}$$

So the number of false positives generally increases. Because

$$\text{Precision} = \frac{TP}{TP + FP}$$

those additional false positives can cause precision to decrease.

PR Curve: The Key Tradeoff

- To increase precision, raise the threshold. Suppose we raise the threshold from 0.5 to 0.7, we become more selective about calling something positive.
- Some false positives become true negatives:

$$\text{FP} \rightarrow \text{TN}$$

which tends to improve precision.

But some true positives also become false negatives:

$$\text{TP} \rightarrow \text{FN}$$

which causes recall to decrease.

So the fundamental threshold tradeoff is:

Lower threshold $\Rightarrow$ Recall $\uparrow$, usually Precision $\downarrow$

Higher threshold $\Rightarrow$ Precision $\uparrow$, usually Recall $\downarrow$

What a good PR curve looks like?

A strong classifier has a curve that stays:

- High (precision close to 1) while also going
- far to the right (recall close to 1).

So the closer the curve is to the top-right, the better.

PR Curve: Baseline

- Unlike ROC curves, PR curves have a baseline that depends on your dataset.

- **Baseline precision = prevalence of positives**

If 5% of your dataset is positive, then the “random guessing” baseline is:

$$\text{Precision} = 0.05$$

- So if your PR curve is mostly above 0.05, you’re doing better than random.
- Many libraries report Average Precision (AP) that is essentially the area under the PR curve:
 - Higher AP = better model
 - AP is more meaningful than ROC-AUC for imbalanced data.

PR Curve: Practical Interpretation Example

If at some threshold, your model has:

- Recall = 0.90 and Precision = 0.20

This means:

- It catches 90% of true positive.
- But 80% of predicted positives are false alarms.

This might be fine in:

- disease screening, fraud detection, and safety alerts

But terrible in:

- expensive interventions and limited resource settings

How to Use the PR Curve?

You typically pick a threshold based on what matters more:

- If missing positives is costly:
→ choose a point with high recall
- If false positives are costly:
→ choose a point with high precision

ROC vs. PR

- Neither curve is universally “better.” They emphasize different aspects of performance.
- ROC is useful for understanding discrimination across both classes.
- PR is especially informative when the positive class is rare and finding positive cases is the primary concern.
- PR analysis focuses attention much more directly on:
When the model says “positive,” how often is it right, and how many true positives does it find?

Overfitting

What is overfitting?

Overfitting occurs when a model learns patterns specific to the training data—including noise—rather than patterns that generalize to new data.

What is overfitting?

Suppose we have a simple nonlinear relationship:

$$y = f(x) + \epsilon$$

where ϵ represents random noise.

- **Underfitting:** A straight line attempts to describe a nonlinear relationship.

$$\hat{y} = \theta_0 + \theta_1 x$$

It cannot capture the underlying pattern and causes:

High training error + high test error

- **Overfitting:** A highly flexible model passes through or near almost every training observation. It causes:

Low training error + high test error

What is overfitting?

- **Appropriate fit:** A moderately flexible curve captures the major biological relationship without trying to reproduce every observation. This leads to:

Low training error + low test error

How to overcome overfitting?

- **Reduce model complexity:** Use a simpler model, shallower decision tree, fewer polynomial terms, fewer neural-network parameters, or fewer features.
- **Regularization:** For logistic regression, regularization discourages excessively large coefficients and reduces model variance.
- **Use proper model evaluation:** Cross-validation helps assess generalization performance and supports model/hyperparameter selection.

Regularization

What is regularization?

- Regularization discourages a model from using excessively large coefficients, making the model less complex and often improving its ability to generalize to new data.
- Consider logistic regression:

$$\log \left(\frac{p}{1-p} \right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d$$

Without regularization, logistic regression chooses the coefficients $\theta_0, \theta_1, \dots, \theta_d$ that best fit the training data. The problem is that the model can sometimes fit the training data **too well**.

Regularization Example

Imagine predicting disease status using three biomarkers:

$$\text{logit}(p) = -1.2 + 0.2x_1 - 0.8x_2 + 18.7x_3$$

The coefficient for x_3 is extremely large. That may mean x_3 is genuinely an extremely strong predictor. But it could also mean that the model has found a peculiar pattern in the training data. For example, perhaps by chance the following happens in the training data:

$x_3 < 1.5 \rightarrow$ almost everyone healthy

$x_3 > 1.5 \rightarrow$ almost everyone diseases

The model may assign x_3 a very large coefficient to exploit this pattern. But the pattern might not hold in new patients. This is **overfitting**.

Regularization Explained

- Regularization modifies the model's objective.
- Without regularization, conceptually the model minimizes: Loss
We ask: How well does the model fit the training observations?
- With regularization, it minimizes: $\text{Loss} + \lambda \times \text{Penalty}$
We ask: How large or complex are the coefficients?
- So the optimizer must make a compromise. A set of coefficients might fit the training data slightly better but require enormous coefficient values. Regularization says:
I will accept a slightly worse training fit if I can obtain a simpler model with smaller coefficients.
That tradeoff is the heart of regularization.

L2 / Ridge Regularization

- A common form of regularization for logistic regression is **L2 regularization**, also called **Ridge regularization**.
- The penalty is based on squared coefficients:

$$\theta_1^2 + \theta_2^2 + \cdots + \theta_d^2$$

So the optimization becomes conceptually

$$\text{Loss} + \lambda \sum_{j=1}^d \theta_j^2$$

- Suppose: $\theta_1 = 1, \theta_2 = 2, \theta_3 = 10$
The L2 penalty is: $1^2 + 2^2 + 10^2 = 105$
That coefficient of 10 contributes: $10^2 = 100$ by itself.
- Therefore, L2 strongly discourages very large coefficients.

L2 / Ridge Regularization

- Imagine the unregularized model produces:

$$[0.8, -1.3, 7.5, 0.2]$$

With increasing L2 regularization, we might see:

$$[0.7, -1.1, 4.2, 0.18]$$

then

$$[0.4, -0.7, 1.8, 0.10]$$

- The coefficients are shrunk toward zero. This is why regularization is also called **shrinkage**.
- Importantly, L2 generally does not make coefficients exactly zero.

L1 / Lasso Regularization

- **L1 regularization**, also called **Lasso regularization**, uses the absolute values of coefficients:

$$|\theta_1| + |\theta_2| + \cdots + |\theta_d|$$

Conceptually: $\text{Loss} + \lambda \sum_{j=1}^d |\theta_j|$

- A major difference between L1 and L2 regularization is that L1 can drive coefficients exactly to zero.

- For example:

No regularization: $[0.8, -1.3, 0.15, 0.07, 2.1]$

L1 regularization: $[0.5, -0.9, 0, 0, 1.4]$

Now coefficients 3 and 4 have been set to zero, effectively excluding those predictors from the model.

- Therefore, **L1 regularization can perform implicit feature selection**. This can be attractive in high-dimensional biological datasets such as gene expression, metabolomics, proteomics, or exposomics.

Elastic Net

- **Elastic Net** combines L1 and L2 regularization with the goal to minimize:

$$\text{Loss} + \lambda \left[\alpha \sum_j |\theta_j| + (1 - \alpha) \sum_j \theta_j^2 \right]$$

where λ controls the strength of regularization and α controls the mixture.

$\alpha = 0 \Rightarrow$ L2 / Ridge regularization

$0 < \alpha < 1 \Rightarrow$ Elastic Net: combines both effects

$\alpha = 1 \Rightarrow$ L1 / Lasso regularization

- So Elastic Net can simultaneously shrink coefficients and eliminate some variables. **Elastic Net can be especially useful when predictors are numerous and correlated.**
- In current scikit-learn terminology, `l1_ratio` plays the role of α .

Regularization: C

- Regularization strength is commonly represented mathematically by λ .
- Scikit-learn's `LogisticRegression` instead uses C , which is inversely related to regularization strength,

$$C \propto \frac{1}{\lambda}$$

- Therefore

small $C \rightarrow$ strong regularization $\rightarrow$ more coefficient shrinkage $\rightarrow$ simpler model

whereas:

large $C \rightarrow$ weak regularization $\rightarrow$ less coefficient shrinkage $\rightarrow$ more flexible model

- In scikit-learn, use `LogisticRegression(C=100)` or `LogisticRegression(C=0.001)`. The latter uses a much stronger regularization.

Concept: Bias and Variance

- Too little regularization: The model becomes flexible:

low bias + high variance

It may fit peculiarities of the training data and cause **overfitting**.

- Too much regularization: The coefficients become excessively constrained and

high bias + low variance

The model cannot capture genuine relationships and causes **underfitting**.

- We want a balance and appropriate regularization. So we need to tune C .

What Exactly Are Bias and Variance?

- Imagine that we could repeatedly draw different training datasets from the same population and fit the model each time.
- For a particular input x , each training dataset would produce a slightly different estimate

$$\hat{f}(x) = \hat{P}(Y = 1|X = x)$$

of the true, unknown probability

$$f(x) = P(Y = 1|X = x)$$

What Exactly Are Bias and Variance?

- If we train B models on B different training datasets, we obtain B estimated probabilities:

$$\hat{f}^{(1)}(x), \hat{f}^{(2)}(x), \dots, \hat{f}^{(B)}(x)$$

The average predicted probability is

$$\overline{\hat{f}(x)} = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x)$$

As B becomes very large,

$$\overline{\hat{f}(x)} \approx E \left[\hat{f}(x) \right]$$

What Exactly Are Bias and Variance?

- We define bias as:

$$\text{Bias}(x) = E \left[\hat{f}(x) \right] - f(x)$$

and **squared bias**:

$$\text{Bias}^2(x) = \left(E \left[\hat{f}(x) \right] - f(x) \right)^2$$

- With the repeated-sampling experiment, $\text{Bias}(x)$ can be approximated as:

$$\text{Bias}(x) \approx \overline{\hat{f}(x)} - f(x)$$

- So high bias means that **even after averaging models trained on many different datasets, the model systematically misses the true relationship.**

What Exactly Are Bias and Variance?

- **Variance** measures how much the fitted model changes from one training dataset to another:

$$\begin{aligned}\text{Variance}(x) &= E \left[\left(\hat{f}(x) - E \left[\hat{f}(x) \right] \right)^2 \right] \\ &\approx \frac{1}{B} \sum_{b=1}^B \left[\hat{f}^{(b)}(x) - \overline{\hat{f}}(x) \right]^2\end{aligned}$$

- So high variance means **the model is sensitive to which particular observations happened to be in the training set.**

Concept:

Bias: How far is the average model from truth?

Variance: How far do individual models vary around the average model?

Can Bias and Variance Be Quantified?

- Yes, for simulated data.
- Let's say we generate 1,000 different training datasets, fit a model to each one and obtain an estimated probability $\hat{f}^{(b)}(x)$ for the same x . If the predictions are:

$$\hat{f}^{(1)}(x), \hat{f}^{(2)}(x), \dots, \hat{f}^{(1000)}(x)$$

we could calculate

$$\bar{\hat{f}}(x) = \frac{1}{1000} \sum_{b=1}^{1000} \hat{f}^{(b)}(x)$$

Then:

$$\text{Bias}^2(x) = \left(\bar{\hat{f}}(x) - f(x) \right)^2$$

and

$$\text{Variance}(x) = \frac{1}{1000} \sum_{b=1}^{1000} \left(\hat{f}^{(b)}(x) - \bar{\hat{f}}(x) \right)^2$$

We could average these quantities over many x 's to obtain an overall estimate.

Can Bias and Variance Be Quantified?

- But for real biological data, we don't know the true $f(x)$.
- As a result, the true bias

$$E \left[\hat{f}(x) \right] - f(x)$$

cannot generally be calculated.

- Variance is somewhat easier to investigate empirically. We can repeatedly resample the training data using cross-validation or bootstrapping, refit the model, and examine how much its predictions or coefficients change.

Bias and variance describe how a model behaves across repeated training samples. In practice, because the true data-generating function is unknown, we typically tune regularization using cross-validation rather than directly measuring bias and variance.

Importance of Predictor Scaling for Regularization

- Suppose two predictors are:
 - Age: 20 - 80
 - Protein level: 0 - 100,000
- The numerical scale affects the size of the coefficients needed to produce a given change in the linear predictor.
- Because regularization penalizes **coefficient magnitude**, predictors measured on different scales can be penalized unfairly.
- Therefore, when using regularized logistic regression, it is generally important to standardize continuous predictors before training a model:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

- The intercept is generally not regularized.

Evaluation of Generalization Performance

Evaluation Protocol

- **Goal:** estimate how well a trained model will generalize to unseen data.
- Two common strategies:
 - Hold-out validation
 - cross-validation (CV)
- **Key idea:** Model development and final model evaluation should be kept separate.

Hold-Out Method

Procedure

- Split the dataset into two parts: training set and test set.
- Train a model on the training set
- Evaluate the final trained model on the test set

Why is it called hold-out?

Because you literally hold out some data and do not let the model see it during training. That held-out portion is used only for evaluation.

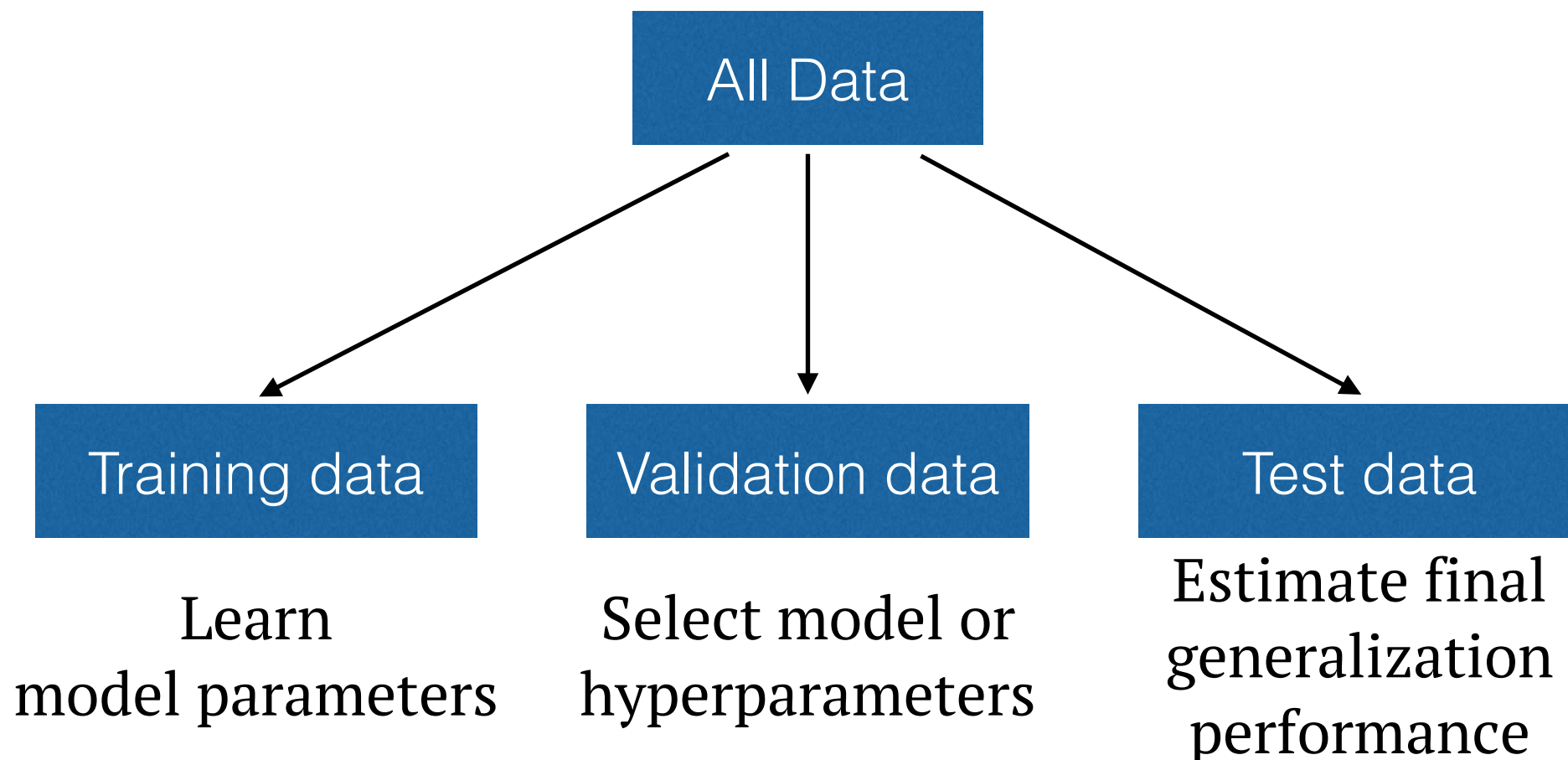
Typical split ratios

- 80% train / 20% test, or 70% train / 30% test

This simple train/test holdout is appropriate when no hyperparameter tuning or model selection is performed using the held-out data.

3-Way Hold-Out

- When hyperparameters need to be tuned, a validation set is needed in addition to the training and test sets.
- Common train/validation/test splits: 70/15/15 or 80/10/10.
- The test data should be used for final evaluation only.
- Using the test data for hyper-parameter tuning causes **data leakage** because information from the test data influences model development.



Important

The test data should not influence model selection.

Question:

Why can't we try five values of C on the test set and simply choose the one with the highest test accuracy?

Workflow for 3-Way holdout

- Step 1: Choose a candidate hyperparameter setting.
- Step 2: Train the model on the training set.
- Step 3: Evaluate it on the validation set.
- Step 4: Repeat Steps 1–3 for different hyperparameter settings and select the best setting based on validation performance.
- Step 5: Retrain the model on training + validation data using the selected hyperparameters.
- Step 6: Evaluate the final model **once** on the test set.

The validation set can be evaluated repeatedly during model development; the test set should be reserved for final evaluation only.

Weakness of 3-way holdout

- The weakness of the 3-way holdout is that your choice of hyperparameter(s) can depend substantially on one particular validation dataset.
- A better approach is **cross-validation**, where different portions of the development data take turns serving as validation data. This usually provides a more stable basis for hyperparameter selection.
- For both holdout and cross-validation, validation data are part of model development, whereas the test data are reserved for final evaluation only.

Cross-Validation

- Cross-validation is a procedure for estimating how well a model will generalize to unseen data by repeatedly training and validating it on different subsets of the development data.
- Instead of relying on one training/validation split, CV averages performance across multiple validation folds.

Development data



Used for cross-validation

while

Test data

Remains untouched.

K-fold Cross-Validation

Procedure

- Split the dataset into K equal-sized folds
- For each fold $i = 1, 2, \dots, K$:
 - Train on the other $K - 1$ folds
 - Validate on fold i .
- Compute the metric on each fold.
- Average the metrics.

K-fold Cross-Validation

Example: 5-fold CV

- Dataset split into 5 folds: A, B, C, D, E
- You run:
 - Train on B+C+D+E, $\rightarrow$ validate on A
 - Train on A+C+D+E, $\rightarrow$ validate on B
 - Train on A+B+D+E, $\rightarrow$ validate on C
 - Train on A+B+C+E, $\rightarrow$ validate on D
 - Train on A+B+C+D, $\rightarrow$ validate on E
- Each sample is used in training 4 times and in validation once.

What does CV Output?

From the K-fold CV, you get:

- Fold scores: 0.81, 0.79, 0.83, 0.80, 0.82
- CV Mean: 0.81
- CV SD: 0.015

This is very useful because:

- Mean = average validation performance
- SD = variability of performance across folds

A low SD suggests more consistent performance across folds. However, because fold scores are not independent, CV SD should not be interpreted as a rigorous uncertainty measure.

What Values of K Should We Use?

General Rule:

- Larger $K \rightarrow$ larger training sets and more model fits/computation
- Smaller $K \rightarrow$ smaller training sets and fewer model fits/computation

$K = 5$ or $K = 10$ are common practical choices.

Leave-One-Out Cross-Validation

Core idea: If you have N samples, LOOCV does cross-validation:

- Hold out 1 sample as the validation sample
- Train the model on the other $N-1$ samples
- Evaluate on the held-out sample
- Repeat this N times, holding out a different sample each time
- Average the performance across all N trials

In one sentence:

LOOCV = k -fold cross validation where $K = N$.

Why Do We Use LOOCV?

- LOOCV uses almost all available data for training each time.
- LOOCV may be useful when the dataset is very small because each model is trained on $N - 1$ observations.

LOOCV: Example

Suppose you have 5 samples as follows:

Sample ID	Label
1	0
2	0
3	1
4	1
5	1

LOOCV: Example

LOOCV runs 5 experiments:

- Experiment 1
 - Train on samples: $\{2, 3, 4, 5\}$
 - Validate on sample: $\{1\}$
- Experiment 2
 - Train on samples: $\{1, 3, 4, 5\}$
 - Validate on sample: $\{2\}$
- Experiment 3
 - Train on samples: $\{1, 2, 4, 5\}$
 - Validate on sample: $\{3\}$
- Experiment 4
 - Train on samples: $\{1, 2, 3, 5\}$
 - Validate on sample: $\{4\}$
- Experiment 5
 - Train on samples: $\{1, 2, 3, 4\}$
 - Validate on sample: $\{5\}$
- Then average the results.

LOO for Accuracy (Classification)

If each fold gives a correct or incorrect prediction:

Fold	Correct?
1	1
2	1
3	0
4	1
5	1

Then

$$\text{LOOCV Accuracy} = \frac{1 + 1 + 0 + 1 + 1}{5} = 0.80$$

So the overall LOOCV accuracy is simply:

$$\frac{\text{number of correctly classified samples}}{\text{total number of samples}}$$

LOOCV

Each fitted model is trained on almost the full development dataset, so its training size closely resembles that of the final model.

Validation methods comparison

Within the **development data**:

Method	fits per hyperparameter configuration	Training fraction	Validation fraction	Typical Use
Holdout	1	$\sim 70 - 80\%$	$\sim 20 - 30\%$	fast model selection
5-fold CV	5	80%	20%	common CV choice
10-fold CV	10	90%	10%	more computation
LOOCV	N_{dev}	$N_{dev}-1$	1 sample	very small datasets

Choose a Validation Strategy

Situation	Reasonable approach
Large dataset	Holdout may be sufficient
Typical ML dataset	5- or 10-fold CV
Classification with imbalanced classes	Stratified K-fold CV
Very small dataset + inexpensive model	Consider LOOCV
Hyperparameter tuning + unbiased final estimate	CV on development data + untouched test set

Thank you!